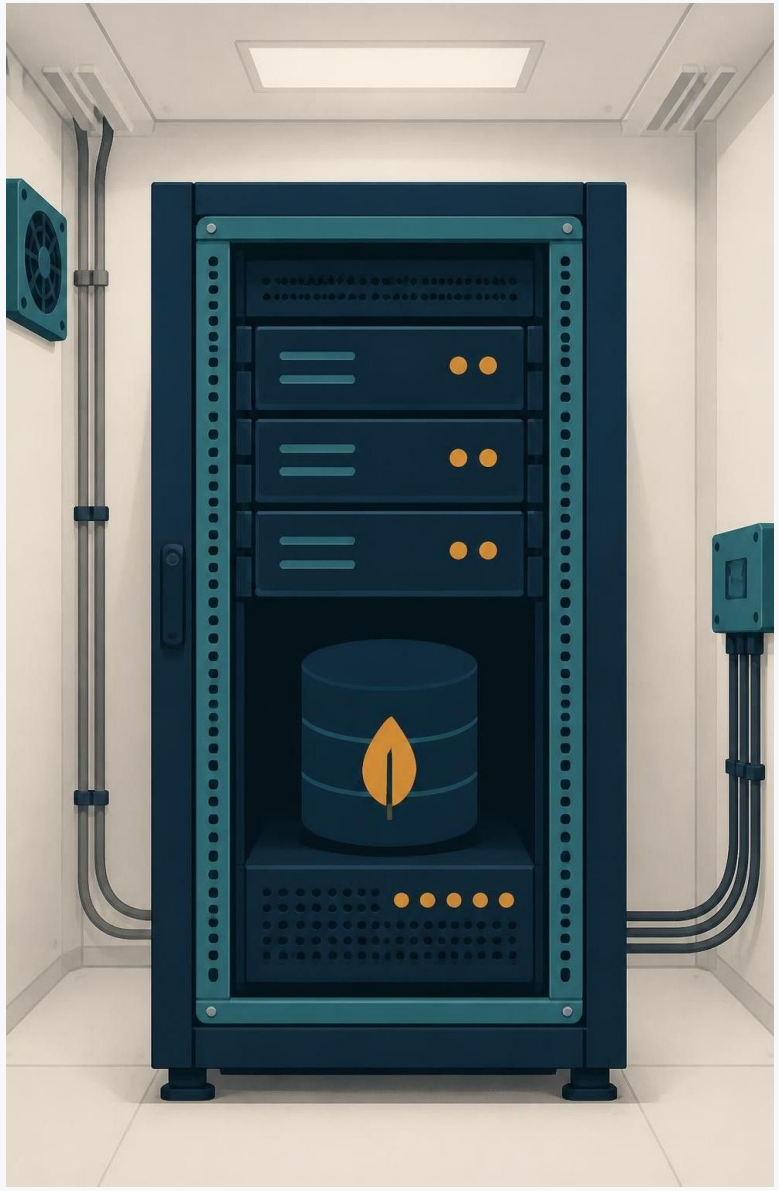
De la Web a una API REST con Spring Boot y MongoDB

Diseño de Software · Cuarto semestre

Pregunta guía: ¿Cómo convertimos un modelo de clases en una API que otros programas puedan utilizar?



¿Qué construiremos hoy?

- Entender cómo se comunican un cliente web y un servidor.
- Reconocer una solicitud y una respuesta HTTP.
- Identificar recursos y operaciones de una API REST.
- Comprender cómo MongoDB conserva documentos.
- Interpretar un modelo de clases y su organización en tres capas.

El profesor realizará la implementación paso a paso.



1. ¿Qué es la Web?

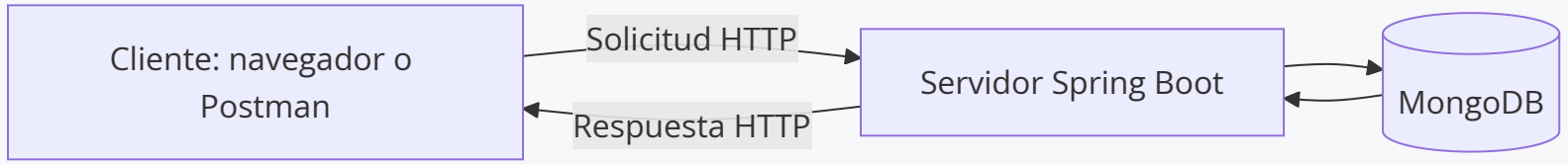
La Web es un sistema de recursos accesibles mediante direcciones (URL), habitualmente sobre Internet.



- **Cliente:** navegador, aplicación móvil o herramienta de pruebas.
- **Servidor:** recibe solicitudes y devuelve respuestas.
- **Recurso:** página, imagen o información de una aplicación.
- **HTTP:** protocolo para intercambiar solicitudes y respuestas.

No son sinónimos: Internet es la red de redes; la Web es uno de los servicios que funciona sobre ella.

Una solicitud viaja y una respuesta regresa



Pregunta: ¿Por qué el navegador no debería conectarse directamente a la base de datos?

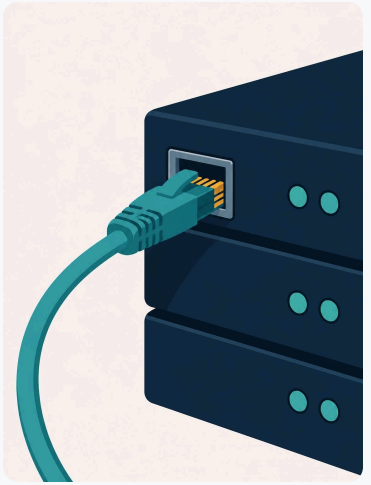
2. HTTP: el lenguaje de la comunicación web

Una petición HTTP contiene, entre otros elementos:

- **Método:** acción solicitada.
- **Ruta o URL:** recurso al que se dirige.
- **Encabezados:** metadatos, como `Content-Type`.
- **Cuerpo:** datos enviados, cuando corresponda.

Una respuesta HTTP contiene:

- **Código de estado:** resultado de la solicitud.
- **Encabezados y cuerpo:** información devuelta.



Métodos HTTP más usados

Diseño: las rutas nombran recursos; el método expresa la operación.

Método	Uso típico	Ejemplo editorial
GET	Consultar	GET /api/libros
POST	Crear	POST /api/libros
PUT	Reemplazar	PUT /api/libros/{id}
PATCH	Modificar parcialmente	PATCH /api/libros/{id}
DELETE	Eliminar	DELETE /api/libros/{id}

Códigos HTTP: ¿qué ocurrió?

200 OK: consulta u operación exitosa.

201 Created: recurso creado.

204 No Content: operación exitosa sin cuerpo.

400 Bad Request: datos de entrada inválidos.

404 Not Found: recurso inexistente.

409 Conflict: conflicto con el estado actual.

500 Internal Server Error: error inesperado del servidor.

Discusión: ¿qué devolveríamos al consultar un libro que no existe?

3. ¿Qué es una API REST?

Una **API** define cómo interactúan dos programas. En una API web, esa interacción puede realizarse mediante HTTP.

REST es un estilo arquitectónico para sistemas distribuidos; en esta clase nos concentraremos en sus ideas introductorias:

- Recursos identificados mediante URI.
- Operaciones usando métodos HTTP.
- Representaciones de recursos, comúnmente JSON.
- Solicitudes autocontenidas: el servidor no depende de recordar el estado conversacional del cliente.



Precisión: usar JSON y GET/POST no garantiza, por sí solo, cumplir todas las restricciones REST.

Ejemplo: recurso «libro»

Solicitud

```
POST /api/libros
Content-Type: application/json

{"titulo":"Diseño de software","isbn":"9780000000000"}
```

Respuesta ilustrativa

```
HTTP/1.1 201 Created
Content-Type: application/json

{"id":"abc123","titulo":"Diseño de software","isbn":"9780000000000"}
```

Pregunta: ¿qué diferencia existe entre la clase `Libro` y su representación JSON?

4. MongoDB: persistencia documental

MongoDB almacena información en **documentos** agrupados en **colecciones**.

- **Documento:** estructura similar a JSON, almacenada internamente como BSON.
- **Colección:** conjunto de documentos.
- **_id:** identificador del documento.
- **Persistencia:** los datos sobreviven a la finalización de una solicitud y, con almacenamiento duradero, al reinicio de la aplicación.



Comparación inicial: colección ≈ tabla; documento ≈ fila. La equivalencia no es exacta.

Documento MongoDB: ejemplo editorial

```
{
  "_id": "abc123",
  "titulo": "Diseño de software",
  "isbn": "9780000000000",
  "anioPublicacion": 2026,
  "autorIds": ["autor01", "autor02"]
}
```

Decisión de diseño: ¿guardamos los autores dentro del documento de libro o mediante referencias?

La respuesta depende de cómo se consultan y modifican los datos; no existe una regla universal.

5. Caso «Editorial»: publicaciones de distintos tipos

Una editorial necesita administrar un catálogo de **publicaciones**, que incluye **libros** y **discos**. Cada publicación comparte datos generales, pero cada tipo tiene información específica.



- Una publicación tiene identificador, título, fecha de publicación y precio.
- Un libro añade ISBN y número de páginas.
- Un disco añade duración y formato.
- El sistema permite registrar, consultar y listar publicaciones; también consultar libros y discos por separado.

Alcance: modelo didáctico propuesto a partir de las clases `Publicacion`, `Libro` y `Disco` mencionadas para el ejercicio. No se han verificado los atributos, rutas ni operaciones exactos del repositorio.

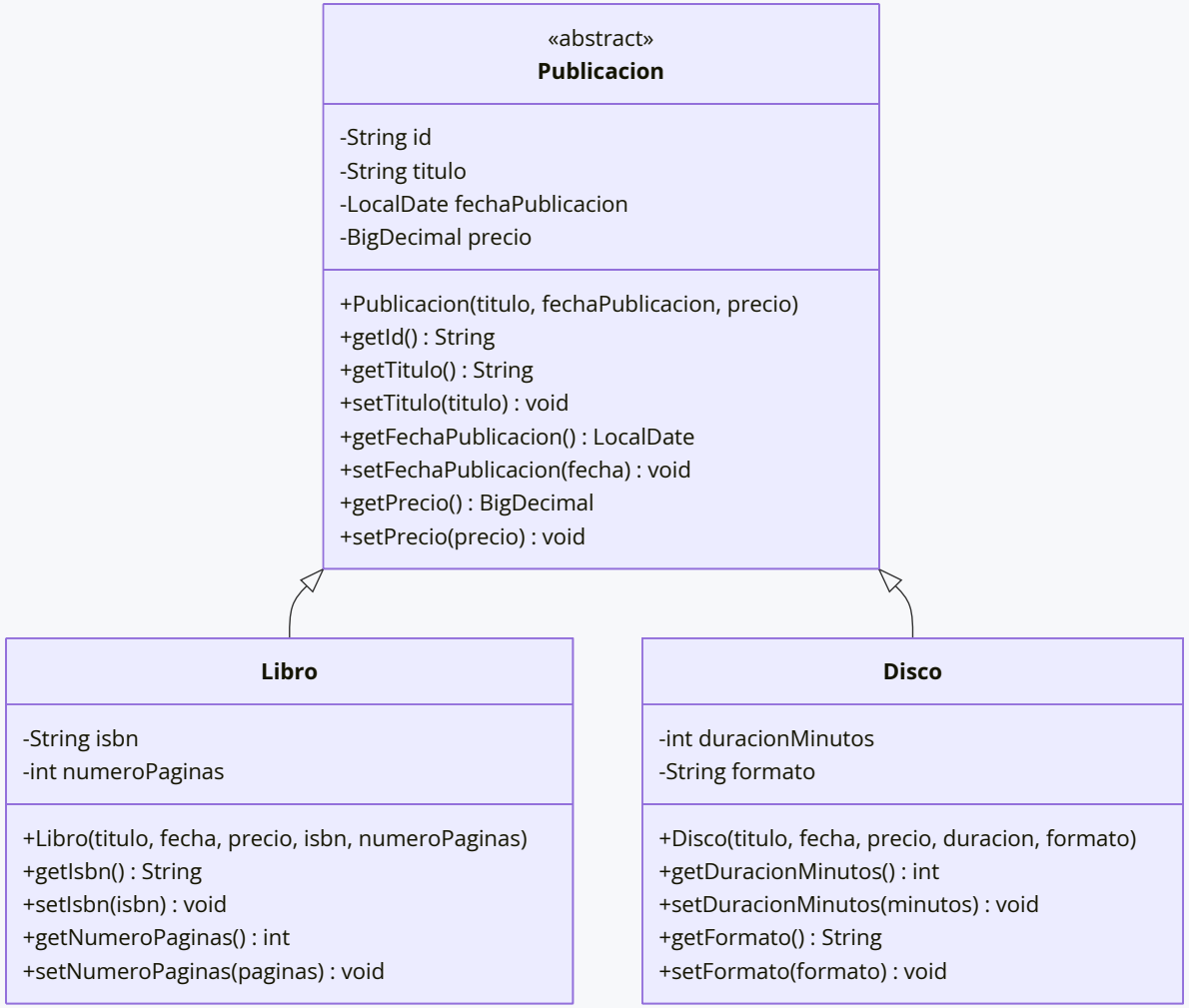
Herencia: ¿qué comparten Libro y Disco?

- `Publicacion` representa lo **común** a los tipos de publicación.
- `Libro` y `Disco` **heredan** sus atributos y operaciones comunes.
- Las especializaciones incorporan sus propios datos.
- Una relación de **generalización UML** se dibuja con línea continua y triángulo blanco apuntando a la superclase.



Pregunta: ¿por qué sería un error dibujar `Libro` — `Publicacion` como asociación si queremos expresar «Libro es una Publicación»?

Diagrama de clases base: Publicación, Libro y Disco



Lectura UML: cada objeto `Libro` o `Disco` es también una `Publicacion`. La herencia **no lleva multiplicidades**: no es una asociación.

Polimorfismo y capacidades de la API

Modelo de dominio: `Publicacion` define lo común; las clases hijas especializan el tipo de publicación.

Posibles recursos REST para el ejercicio:

Advertencia: son rutas propuestas para explicar el diseño, no endpoints verificados en GitHub.

Método	Ruta ilustrativa	Capacidad
GET	/api/publicaciones	Listar publicaciones de ambos tipos
GET	/api/publicaciones/{id}	Consultar una publicación
POST	/api/libros	Registrar un libro
POST	/api/discos	Registrar un disco
GET	/api/libros	Listar libros
GET	/api/discos	Listar discos

Ejemplos JSON: mismo concepto, tipos distintos

Libro

```
{
  "tipo": "LIBRO",
  "titulo": "Diseño orientado a objetos",
  "precio": 58000,
  "isbn": "9780000000000",
  "numeroPaginas": 320
}
```

Disco

```
{
  "tipo": "DISCO",
  "titulo": "Concierto en vivo",
  "precio": 42000,
  "duracionMinutos": 74,
  "formato": "CD"
}
```

Pregunta: ¿qué campos comparten ambos documentos? ¿Cuáles son propios de cada subtipo?

MongoDB: una colección con tipos de publicación

Estrategia didáctica posible: almacenar ambos subtipos en la colección `publicaciones`, usando un campo discriminador `tipo`.

```
{
  "_id": "pub-102",
  "tipo": "LIBRO",
  "titulo": "Diseño orientado a objetos",
  "precio": 58000,
  "isbn": "9780000000000",
  "numeroPaginas": 320
}
```

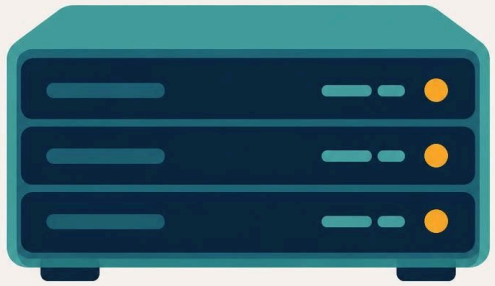
También podrían utilizarse colecciones separadas. **La elección es una decisión de persistencia**, no algo que se deduzca automáticamente de la herencia UML.

Del modelo del dominio a responsabilidades de aplicación

El diagrama de clases responde **qué objetos existen, qué comparten y cómo se especializan**.

Para publicar el modelo como API debemos decidir:

1. ¿Qué clase recibe `POST /api/libros` y `POST /api/discos`?
2. ¿Dónde se comprueban los datos y reglas de cada tipo?
3. ¿Qué componente persiste y recupera las publicaciones?
4. ¿Cómo se devuelve la información de una publicación concreta sin perder su tipo?



Clave: agregar capas no significa convertir las entidades de dominio en controladores o repositorios.

6. Diagrama introductorio de tres capas

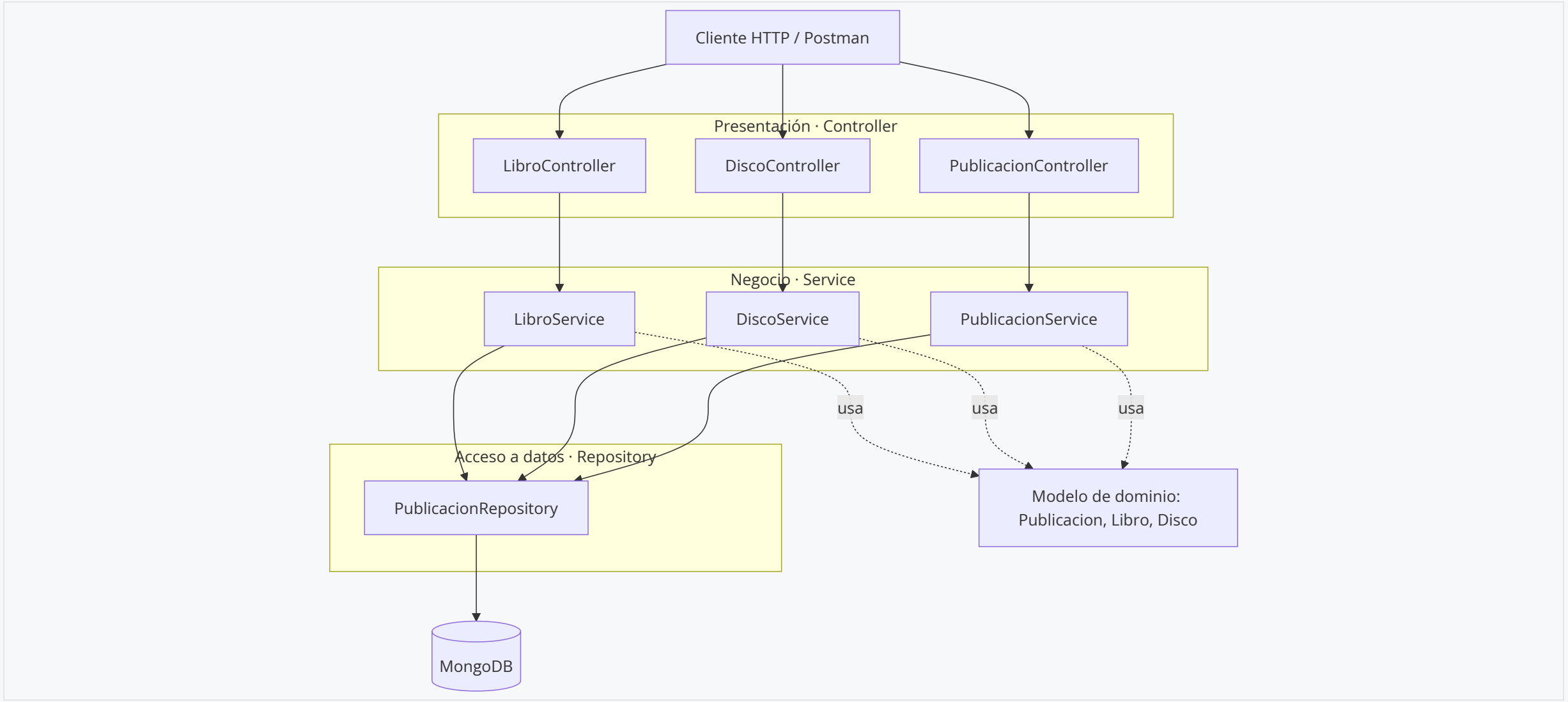
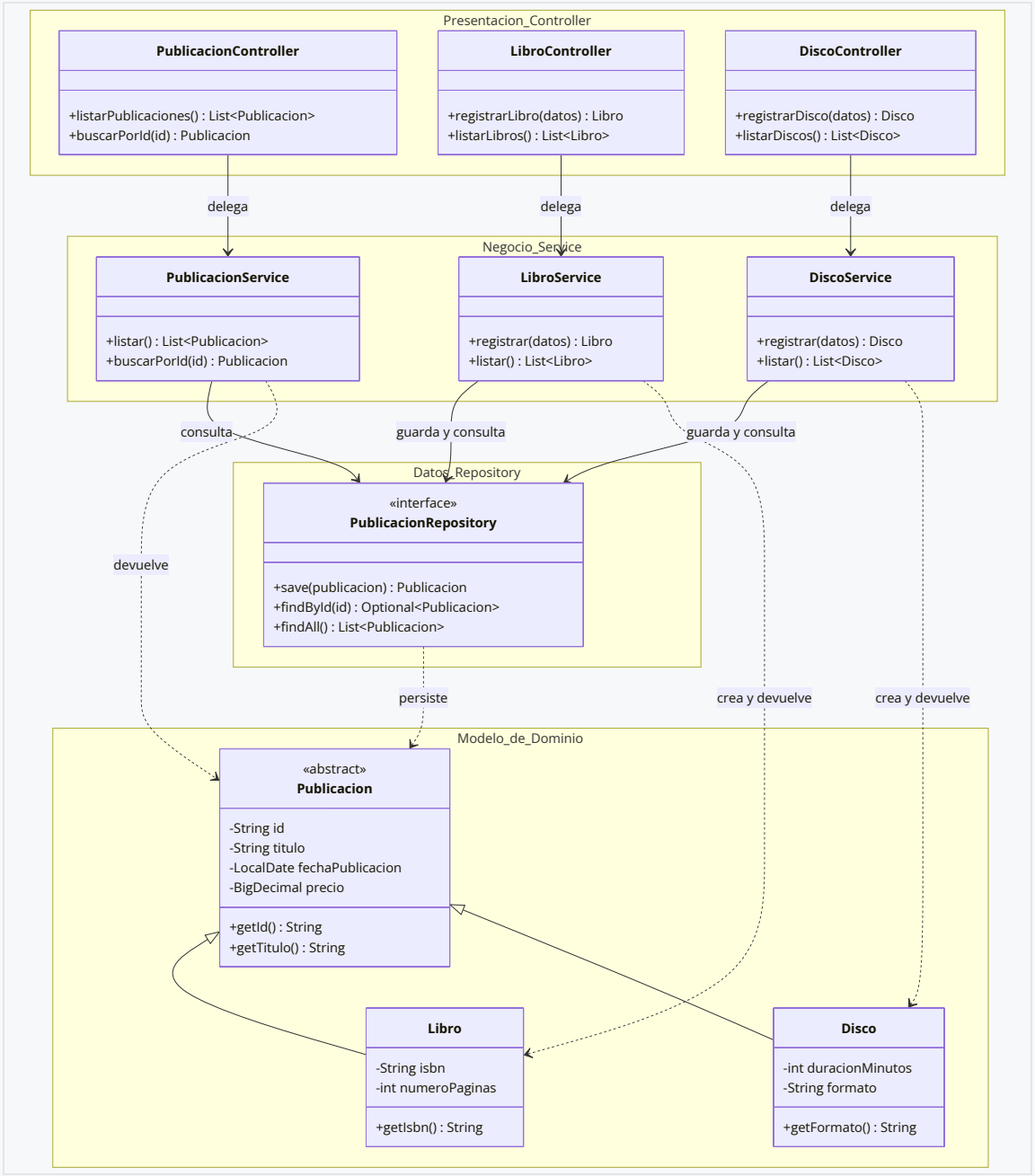


Diagrama conceptual ilustrativo: Controller → Service → Repository. Las entidades de dominio se representan aparte; **no constituyen una cuarta capa**.

Diagrama UML de clases: implementación organizada en tres capas



Cómo leer el diagrama UML de tres capas

Cómo leerlo: las flechas continuas entre Controller, Service y Repository representan asociaciones de colaboración (referencias mantenidas mediante inyección de dependencias); las flechas punteadas hacia el modelo indican uso de tipos en operaciones. El triángulo hueco indica herencia. `Publicacion`, `Libro` y `Disco` son entidades del dominio, **no una cuarta capa**.

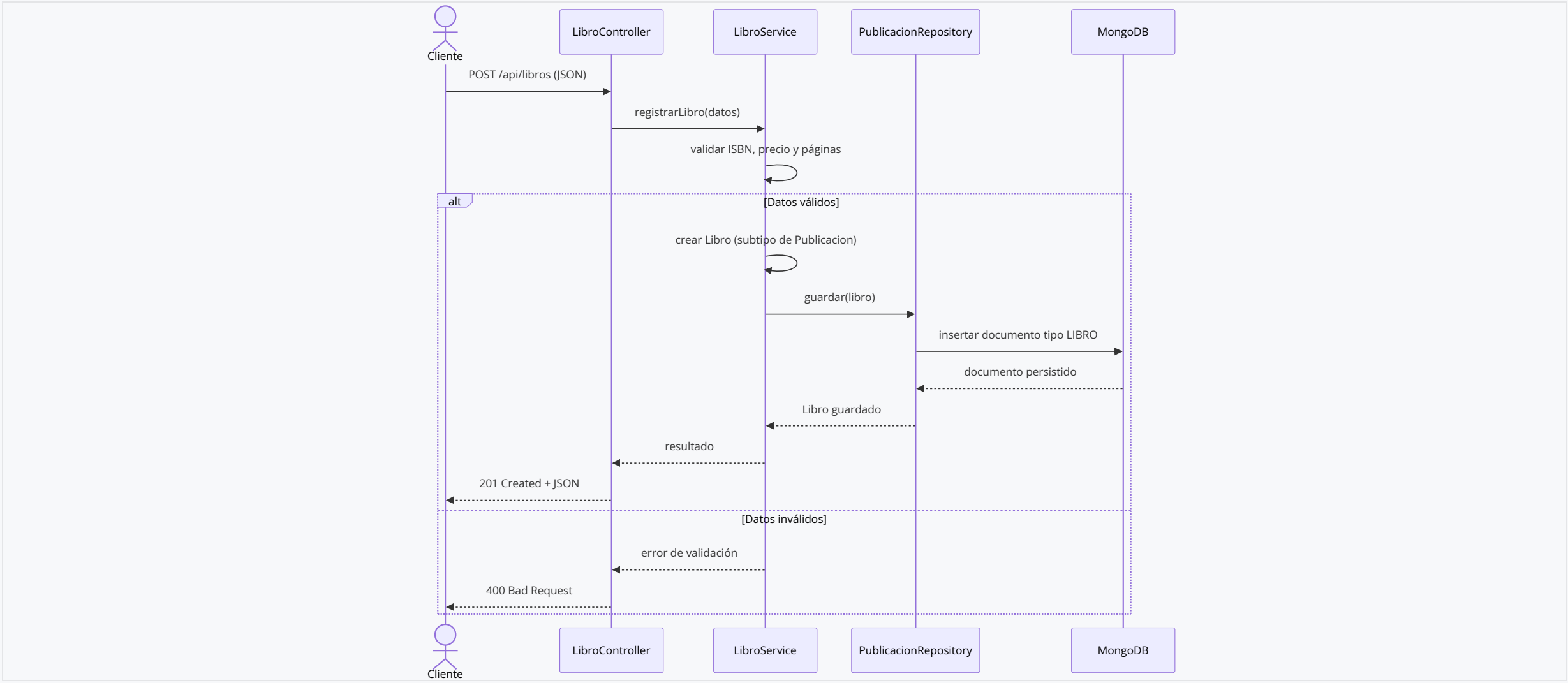
Alcance: diseño didáctico propuesto, no inventario verificado del repositorio. Los métodos están simplificados para visualizar responsabilidades; en una API real pueden utilizarse DTO y tipos HTTP en los controladores. MongoDB es el almacenamiento externo que utiliza la implementación del repositorio, no una clase de dominio.

¿Qué responsabilidad tiene cada capa?

Principios de diseño: alta cohesión, bajo acoplamiento y responsabilidades claras.

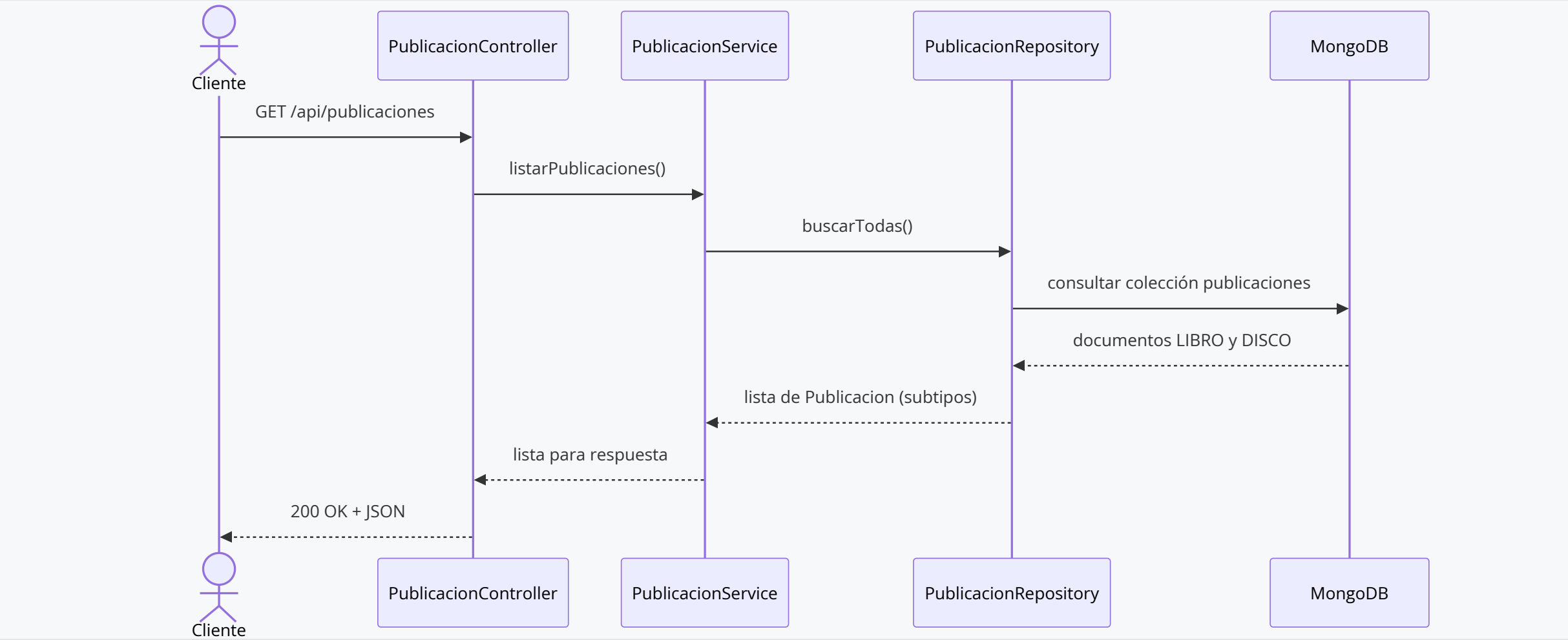
Capa	Pregunta que responde	Ejemplo
Controller	¿Cómo entra y sale la información por HTTP?	Recibe POST /api/libros o /api/discos
Service	¿Qué reglas y pasos requiere el caso de uso?	Valida y coordina el registro del tipo de publicación
Repository	¿Cómo se consulta o almacena información?	Guarda una publicación en MongoDB

Secuencia 1: registrar un libro con la API



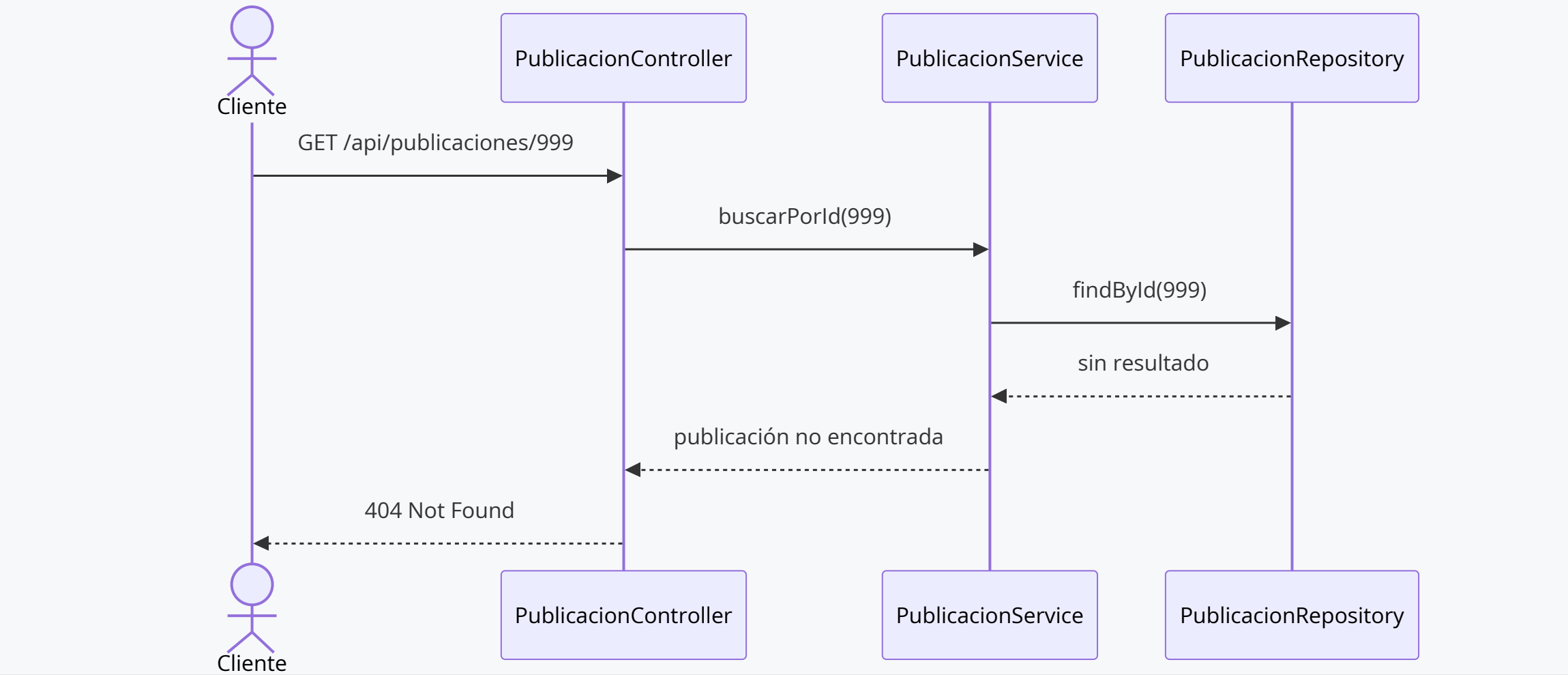
Idea clave: la petición recorre las tres capas; la entidad `Libro` conserva su identidad como subtipo de `Publicacion`.

Secuencia 2: consultar publicaciones de distintos tipos



Discusión: ¿cómo reconoce la aplicación qué documentos corresponden a `Libro` y cuáles a `Disco`? ¿Qué papel cumple el discriminador `tipo`?

Secuencia 3: consultar un recurso inexistente



Pregunta: ¿por qué devolver 404 es más informativo que responder 200 con una publicación vacía?

Capacidades de una API: más allá del camino feliz

- **Crear:** `POST /api/libros` o `POST /api/discos` → 201 si se crea correctamente.
- **Consultar:** `GET /api/publicaciones/{id}` → 200 o 404.
- **Listar:** `GET /api/publicaciones` → 200 y una colección, posiblemente vacía.
- **Validar:** datos inválidos → 400.
- **Persistir:** el Repository guarda y consulta en MongoDB.
- **Representar:** JSON transporta los datos; el modelo UML define los tipos del dominio.



Reto: proponer el flujo de `DELETE /api/publicaciones/{id}` sin escribir código.

Actividad de cierre: pensar antes de programar

En parejas (10 minutos):

1. Propongan los endpoints para crear, listar y consultar libros, discos y publicaciones.
2. Asignen cada operación a Controller, Service y Repository.
3. Expliquen por qué Controller no debería almacenar directamente en MongoDB.
4. Identifiquen una regla de negocio de `Libro` y otra de `Disco` que deban validarse en Service.



Entrega: tabla de responsabilidades y justificación breve.

Puente hacia la demostración del profesor

Secuencia de implementación que observarán:

1. Crear proyecto Spring Boot.
2. Representar `Publicacion` y sus subclases `Libro` y `Disco`.
3. Incorporar persistencia con MongoDB.
4. Implementar Repository → Service → Controller.
5. Probar peticiones HTTP y respuestas JSON.



Criterio de éxito: explicar por qué existe cada clase, no solo conseguir que la API funcione.

Fuentes y alcance

- **Caso Editorial:** modelo pedagógico con herencia `Publicacion` → `Libro` / `Disco` basado en los nombres aportados por el docente; atributos, rutas y persistencia exactos pendientes de contrastar con el código del repositorio.
- **HTTP:** documentación de referencia de MDN, **https://developer.mozilla.org/en-US/docs/Web/HTTP**.
- **REST con Spring:** guía oficial, **https://spring.io/guides/tutorials/rest**.
- **Spring Data MongoDB:** documentación oficial, **https://docs.spring.io/spring-data/mongodb/reference/**.
- **MongoDB:** documentación oficial, **https://www.mongodb.com/docs/manual/**.

Repositorio de referencia: **https://github.com/migueltovarb/api-spring-boot/tree/main/editorial/src** (contenido no verificado en esta edición).

- **Enfoque curricular:** introducción a capas para implementar un diseño orientado a objetos; no es un curso de arquitectura avanzada.